

闪电演讲 | Goodput 是你所需的一切

基于 SLO 的 LLM 推理服务基准测试

Lightning Talk: Goodput is All You Need: SLO-based LLM Inference Benchmarking

郑宇宸

软件工程师

2025/11/15

Yu-Chen Cheng, Software Engineer





郑宇宸 Yu-Chen Cheng

Software Engineer

Focus on building AI/LLM infrastructure based on Kubernetes

Previously **@Heywhale @WizardQuant**
Incoming **SDE @ Tencent Cloud** CSIG TI-ONE

CONTENT

目录

01 Background

02 LLM Inference Benchmarking

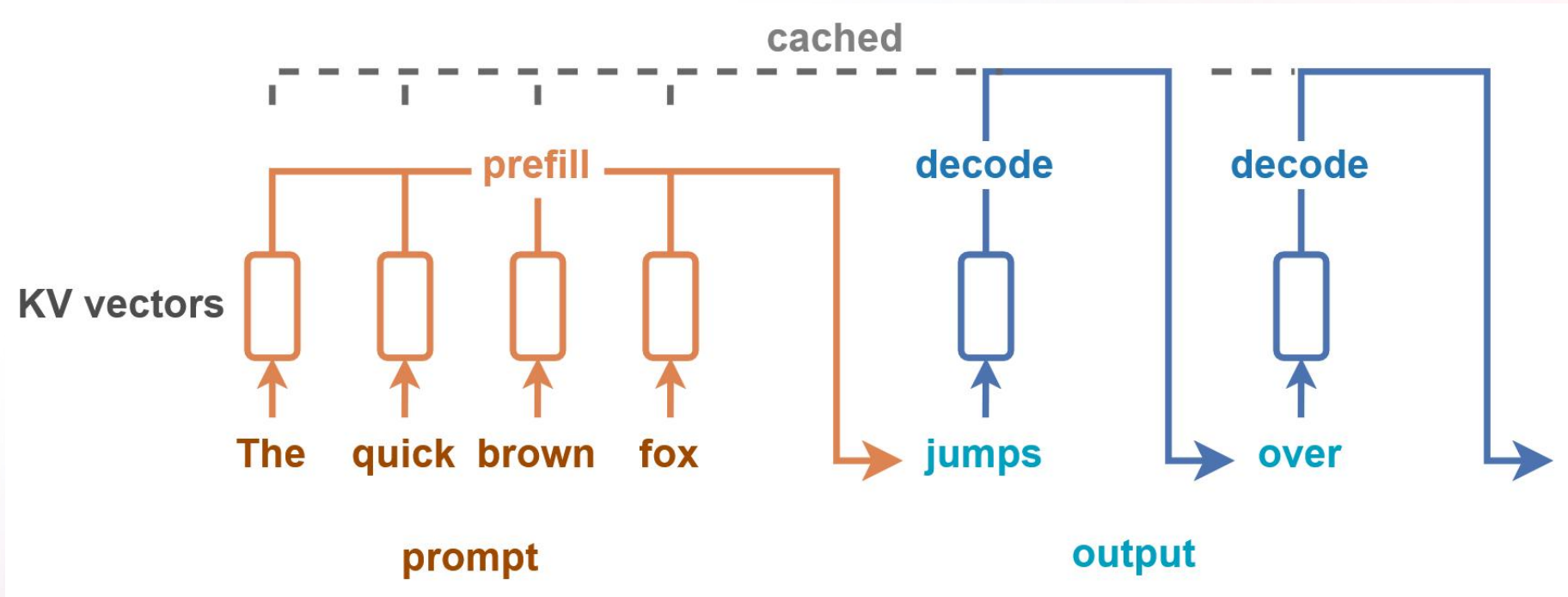
03 Takeaways



Background

LLM Inference and Benchmarking

LLM Inference



Prefill: Computation-intensive

Decode: Memory-intensive

(Source: <https://huggingface.co/blog/tngtech/llm-performance-prefill-decode-concurrent-requests>)

LLM Inference Stack on Kubernetes

Orchestration



vllm-project/aibrix



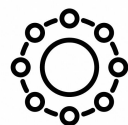
llm-d



NVIDIA Dynamo

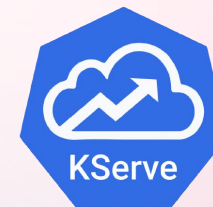


KV Cache
Management

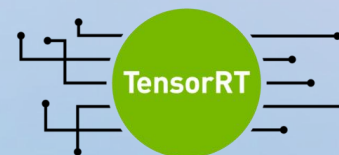


Mooncake

LMCache



Inference

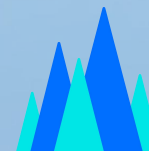


Resource
Management



Compute

On Premise



LLM Inference Stack on Kubernetes

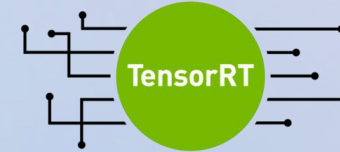
Inference



vLLM



SGL



TensorRT

Frameworks

- vLLM
- SGLang
- TensorRT LLM
- ...

Parameter Tuning

- Tensor Parallelism
- Data Parallelism
- Expert Parallelism
- ...

Optimization

- Speculative Decoding
- Flash Attention
- ...

LLM Inference Stack on Kubernetes

KV Cache Management



LMCache

KV Cache Offloading

KV Cache Sharing

LLM Inference Stack on Kubernetes

Orchestration



vllm-project/aibrix



llm-d



NVIDIA Dynamo



Routing and Load Balancing

- Prefix-Aware Routing
- Fairness Routing
- SLO-Aware Routing
- ...

PD Disaggregation

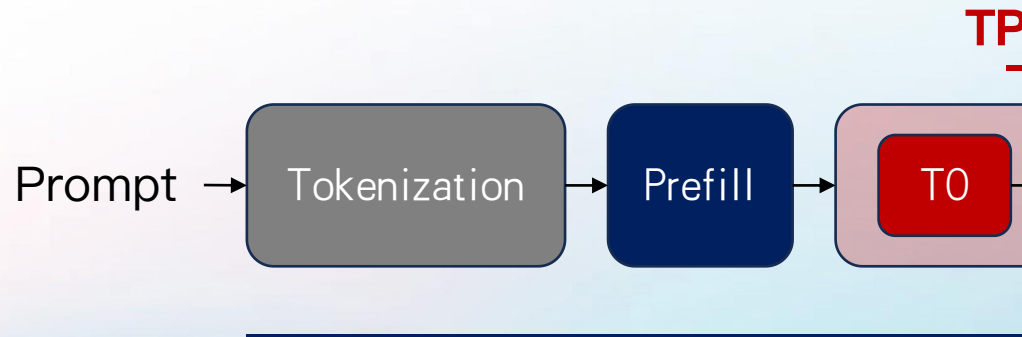
Autoscaling

LLM Inference Metrics

TTFT

Time to First Token. Initial response time when the first output token is generated.

$$TTFT = \text{time_at_first_token} - \text{start_time}$$



Glossary	Meaning	Calculation Formula	Units
TTFT	Time to First Token. Initial response time when the first output token is generated. This is also known as the latency for the input (input) stage.	$TTFT = \text{time_at_first_token} - \text{start_time}$	seconds
End-to-End Latency	End-to-End latency. This metric indicates how long it takes from submitting a query to receiving the full response, including network latencies.	$\text{e2e_latency} = \text{end_time} - \text{start_time}$	seconds
TPOT	Time Per Output Token. The average time between two subsequent generated tokens.	$TPOT = (\text{e2e_latency} - TTFT) / (\text{num_output_tokens} - 1)$	seconds
Output Latency	Output latency. This metric indicates how long it takes to receive the full response after the first token is generated.	$\text{output_latency} = \text{e2e_latency} - TTFT$	seconds
Output Inference Speed	The rate of how many tokens the model can generate per second for a single request.	$\text{inference_speed} = 1 / TPOT$	tokens/second
Num of Input Tokens	Number of prompt tokens.	$\text{num_input_tokens} = \text{tokenizer.encode(prompt)}$	tokens
Num of Output Tokens	Number of output tokens.	$\text{num_output_tokens} = \text{num_completion_tokens}$	tokens
Num of Request Tokens	Total number of tokens processed in one request.	$\text{num_request_tokens} = \text{num_input_tokens} + \text{num_output_tokens}$	tokens
Input Throughput	The overall throughput of input (input process).	$\text{input_throughput} = \text{num_input_tokens} / TTFT$	tokens/second
Output Throughput	The throughput of output (output generation) for a single request.	$\text{output_throughput} = (\text{num_output_tokens} - 1) / \text{output_latency}$	tokens/second

(Source: <https://github.com/sgl-project/genai-bench>)

Service Level Objective (SLO)

Chat Applications

P99 TTFT $\leq$ 200ms

P99 ITL $\leq$ 50ms

Offline Batch

Max Throughput $\geq$ 100 RPS

Retrieval-Augmented Generation (RAG)

P99 TTFT $\leq$ 300ms

P99 ITL $\leq$ 100ms

P99 Request Latency $\leq$ 3s

Code Completion

P99 Request Latency $\leq$ 2s

Pareto Frontier + SLO Attainment

Example: LLM Performance Explorer by BentoML

Model:

nvidia/Llama-3.1-8B-Instruct-FP8

GPU:

H100 × 8

Input Tokens → Output Tokens:

2000 → 2000

Don't see your setup here? [Book a 15-minute custom evaluation.](#)

Latency:

Prefilling Rate (tokens/s)

Throughput:

Output Throughput (tokens/s)

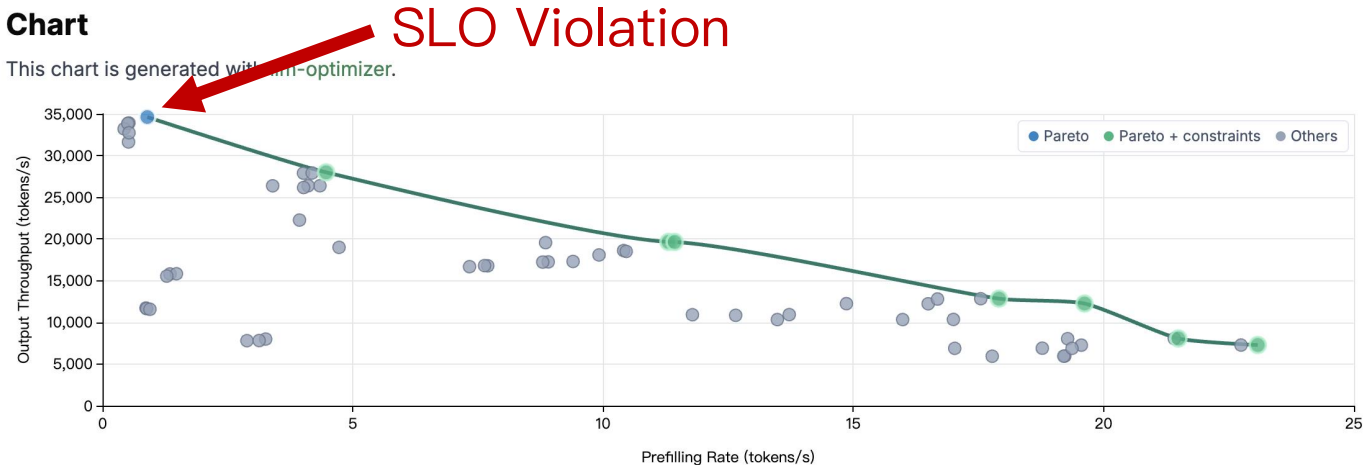
Constraints:

Custom:

P99 ITL <= 50

Chart

This chart is generated with [llm-optimizer](#).



Best Configuration

Output Throughput: 27990.9 tokens/s

With Constraints • Max Concurrency: 256

```
python3 -m sglang.launch_server --model-path nvidia/Llama-3.1-8B-Instruct-FP8 --host 127.0.0.1 --port 30000 --tp-size=2 --dp
```

(Source: <https://www.bentoml.com/llm-perf/> / <https://github.com/bentoml/llm-optimizer>)

What is Goodput?

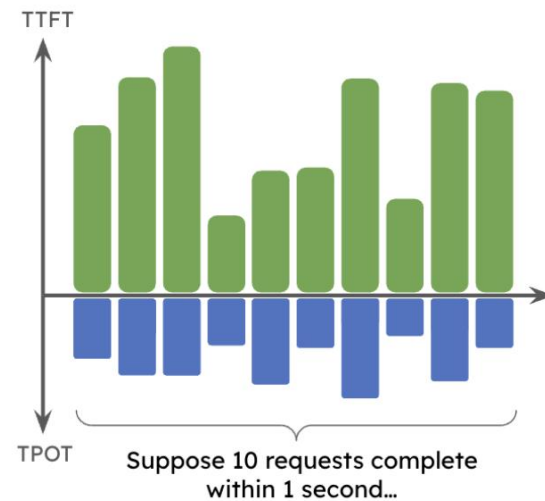
What is Goodput?

Goodput measures the number of completed requests that meet the **SLOs**.

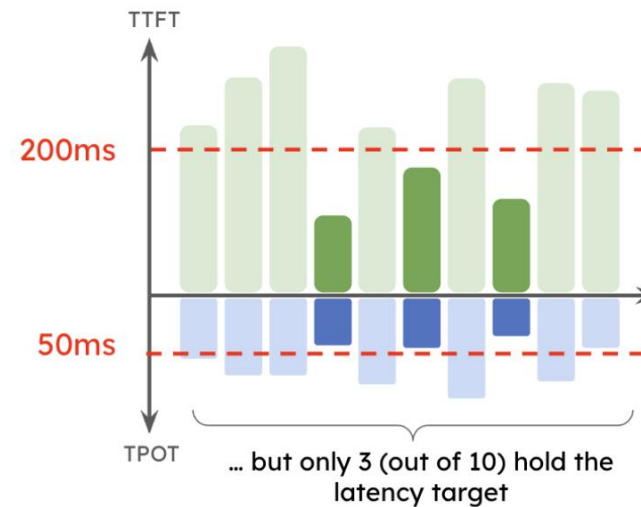
What is Goodput?

High **Throughput** $\neq$ High **Goodput**

Throughput = completed request / time
= **10** req / s



Goodput = completed requests **within SLO** / time
= **3** req / s

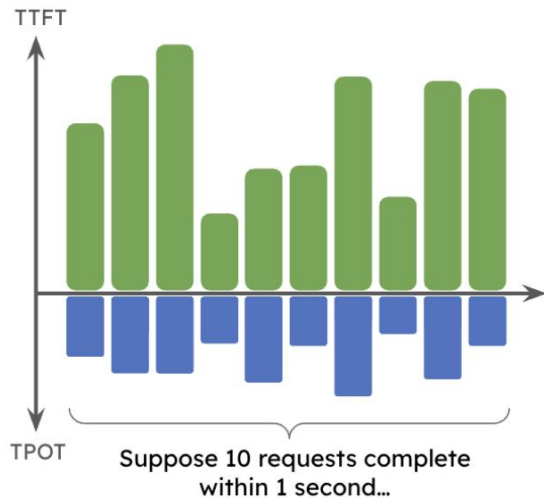


Systems optimizing **throughput** can have low **goodput** under certain SLO constraints.

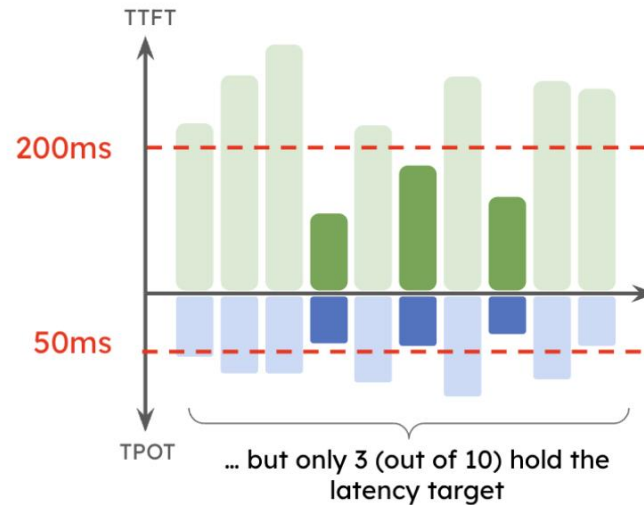
What is Goodput?

High **Throughput** $\neq$ High **Goodput**

Throughput = completed request / time
= **10** req / s



Goodput = completed requests **within SLO** / time
= **3** req / s



GOAL

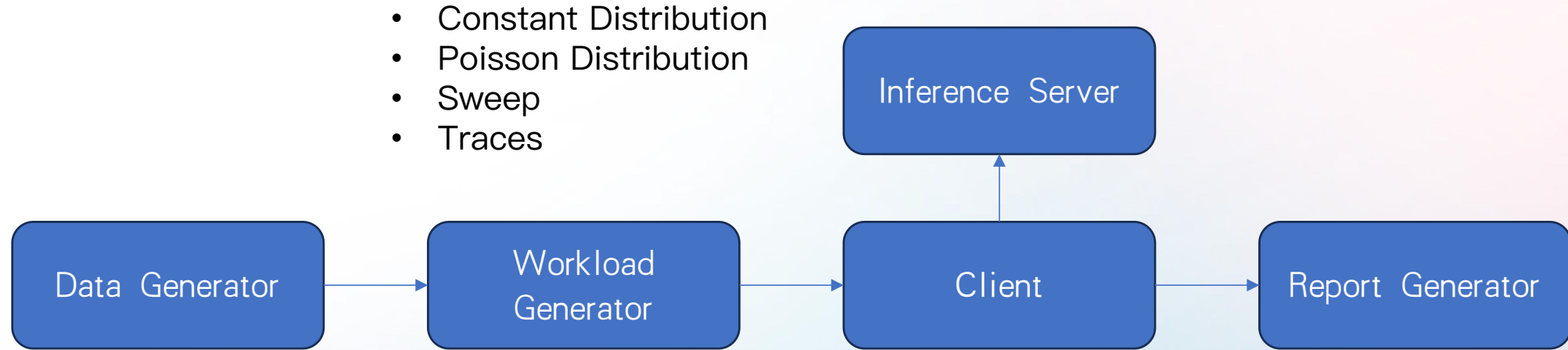
Maximize Goodput

Minimize Instances

LLM Inference Benchmarking

Is Goodput All You Need?

LLM Inference Benchmarking Framework



- Public Datasets
- Synthetic Data
- Logs (Prometheus)
- Traces
 - Mooncake (FAST 2025)
 - Azure (HPCA 2025)
 - Bailian (USENIX ATC 2025)

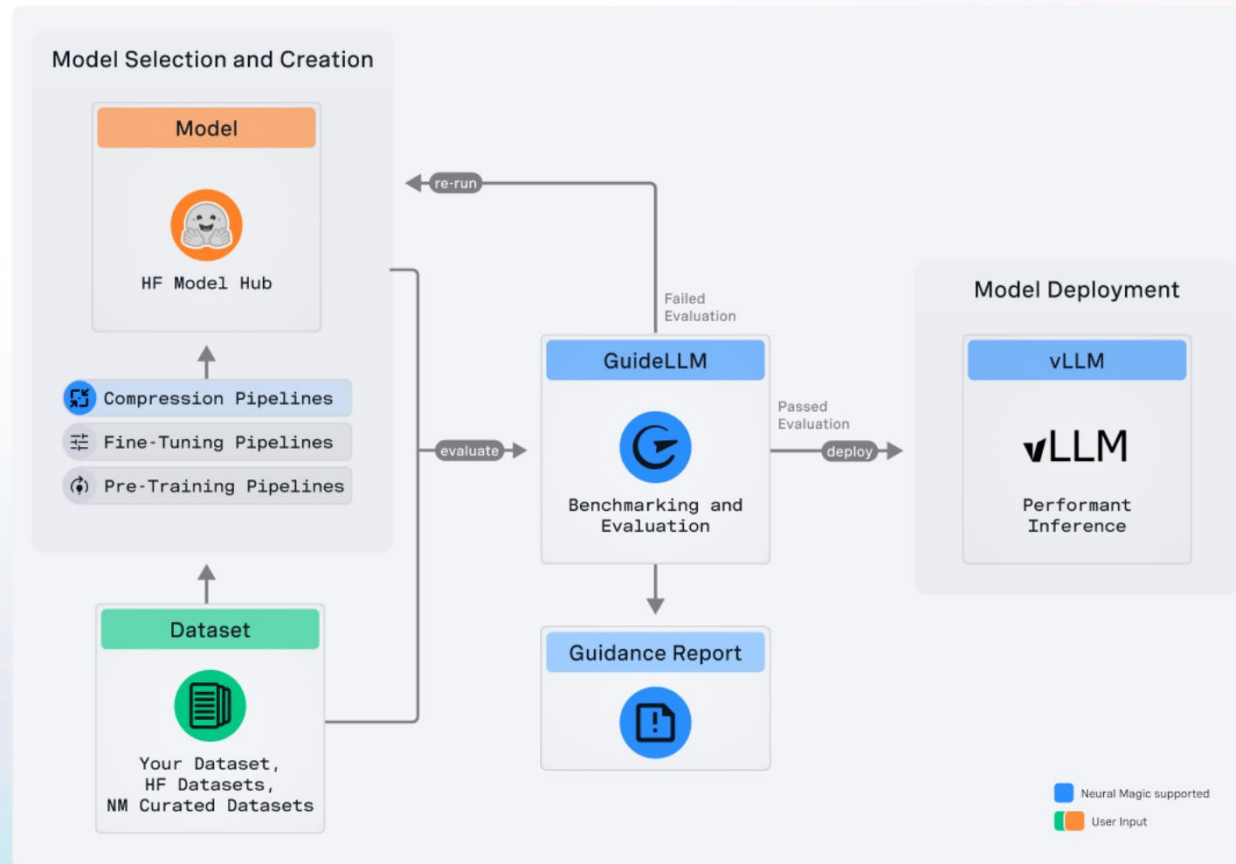
Mooncake Traces

JSON

```
{"timestamp": 0, "input_length": 6758, "output_length": 500, "hash_ids": [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13]}  
{"timestamp": 0, "input_length": 7322, "output_length": 490, "hash_ids": [0, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27]}
```

GuideLLM

Unified, accurate, and beautiful LLM Benchmarking



(Source: <https://github.com/vllm-project/guidellm>)

GuideLLM

Unified, accurate, and beautiful LLM Benchmarking

```
Benchmarks Metadatas:
Run id:f4a92c1-9a1d-4c83-b237-83fcc7971bd3
Duration:117.5 seconds
Profile:type=sweep, strategies=['synchronous', 'throughput', 'constant', 'constant', 'constant', 'constant', 'constant', 'constant', 'constant', 'constant']
Args:max_number=None, max_duration=10.0, warmup_number=None, warmup_duration=None, cooldown_number=None, cooldown_duration=None
Worker:type='generative_requests_worker' backend_type='openai_http' backend_target='http://192.168.4.13:8000' backend_model='neuralmagic/Qwen2.5-7B-quantrization' organization=None, project=None, text_completions_path='/v1/completions', chat_completions_path='/v1/chat/completions'
Request Loader:type='generative_request_loader' data={'prompt_tokens': 128, "output_tokens": 64} data_args=None processor='neuralmagic/Qwen2.5-7B-quantrization' Extras=None
```

Benchmarks Info:

Metadata	Benchmark	Start Time	End Time	Duration (s)	Requests Made Comp Inc Err	Prompt Tok/Req Comp Inc Err	Output Tok/Req Comp Inc Err	Prompt Tok Total Comp Inc Err	Output Tok Total Comp Inc Err
synchronous	throughput	14:42:05	14:42:15	10.0	6 1 0	128.0 128.0 0.0	64.0 43.0 0.0	768 128 0	384 43 0
constant@2.20		14:42:16	14:42:26	10.0	126 510 4	128.1 128.1 0.0	64.0 36.9 0.0	16144 65308 0	8064 7979 0
constant@3.74		14:42:33	14:42:43	10.0	20 4 0	128.0 128.0 0.0	64.0 37.0 0.0	2560 512 0	1280 111 0
constant@5.27		14:42:44	14:42:54	10.0	34 6 0	128.0 128.0 0.0	64.0 29.0 0.0	4354 768 0	2175 174 0
constant@6.81		14:42:55	14:43:05	10.0	48 9 0	128.0 128.0 0.0	63.9 32.9 0.0	6146 1152 0	3068 296 0
constant@8.34		14:43:06	14:43:16	10.0	62 12 0	128.0 128.1 0.0	64.0 32.5 0.0	7938 1537 0	3966 357 0
constant@9.88		14:43:18	14:43:28	10.0	76 15 0	128.1 128.2 0.0	63.9 31.7 0.0	9735 1923 0	4859 444 0
constant@11.41		14:43:29	14:43:39	10.0	88 19 0	128.0 128.1 0.0	63.9 31.1 0.0	11266 2433 0	5627 590 0
constant@12.95		14:43:40	14:43:50	10.0	101 24 0	128.1 128.2 0.0	64.0 32.2 0.0	12934 3077 0	6461 740 0
constant@12.95		14:43:51	14:44:01	10.0	113 28 0	128.1 128.0 0.0	64.0 31.7 0.0	14475 3585 0	7230 855 0

Benchmarks Stats:

Metadata	Request Stats Per Second Concurrency	Out Tok/sec mean	Tot Tok/sec mean	Req Latency (ms) mean median p99	TTFT (ms) mean median p99	ITL (ms) mean median p99	TPOT (ms) mean median p99
synchronous	0.67 1.00	42.7	213.1	1.49 1.49 1.50	48.0 49.0 49.9	22.9 22.9 23.1	22.6 22.6 22.7
throughput	12.95 113.50	828.5	4133.1	8.77 8.65 9.66	1198.8 1112.2 2057.8	120.1 124.6 118.2	118.2 118.2 122.7
constant@2.20	2.06 3.22	131.9	657.9	1.56 1.56 1.58	58.7 58.3 77.0	23.8 23.8 23.9	23.4 23.4 23.5
constant@3.74	3.44 5.52	220.1	1098.4	1.60 1.60 1.64	58.3 56.2 81.0	24.5 24.5 24.9	24.1 24.1 24.5
constant@5.27	4.90 8.21	313.3	1564.0	1.67 1.67 1.73	62.5 59.6 103.4	25.6 25.6 26.4	25.2 25.2 25.9
constant@6.81	6.25 10.88	399.4	1992.3	1.74 1.73 1.83	64.6 61.1 129.3	26.6 26.5 27.5	26.2 26.1 27.0
constant@8.34	7.62 14.21	487.2	2432.2	1.86 1.85 1.97	73.3 63.9 168.2	28.5 28.3 29.3	28.0 27.9 28.9
constant@9.88	8.86 17.64	566.2	2825.0	1.99 1.98 2.08	77.0 65.8 180.4	30.4 30.4 31.2	29.9 29.9 30.7
constant@11.41	10.18 21.23	650.9	3247.1	2.09 2.07 2.24	81.1 67.3 222.1	31.8 31.7 33.8	31.3 31.2 33.3
constant@12.95	11.36 24.92	727.0	3627.1	2.19 2.17 2.37	87.4 70.1 257.3	33.4 33.2 35.9	32.9 32.7 35.3

Saving benchmarks report...

Benchmarks report saved to /home/mark/github/neuralmagic/guidellm/benchmarks.json

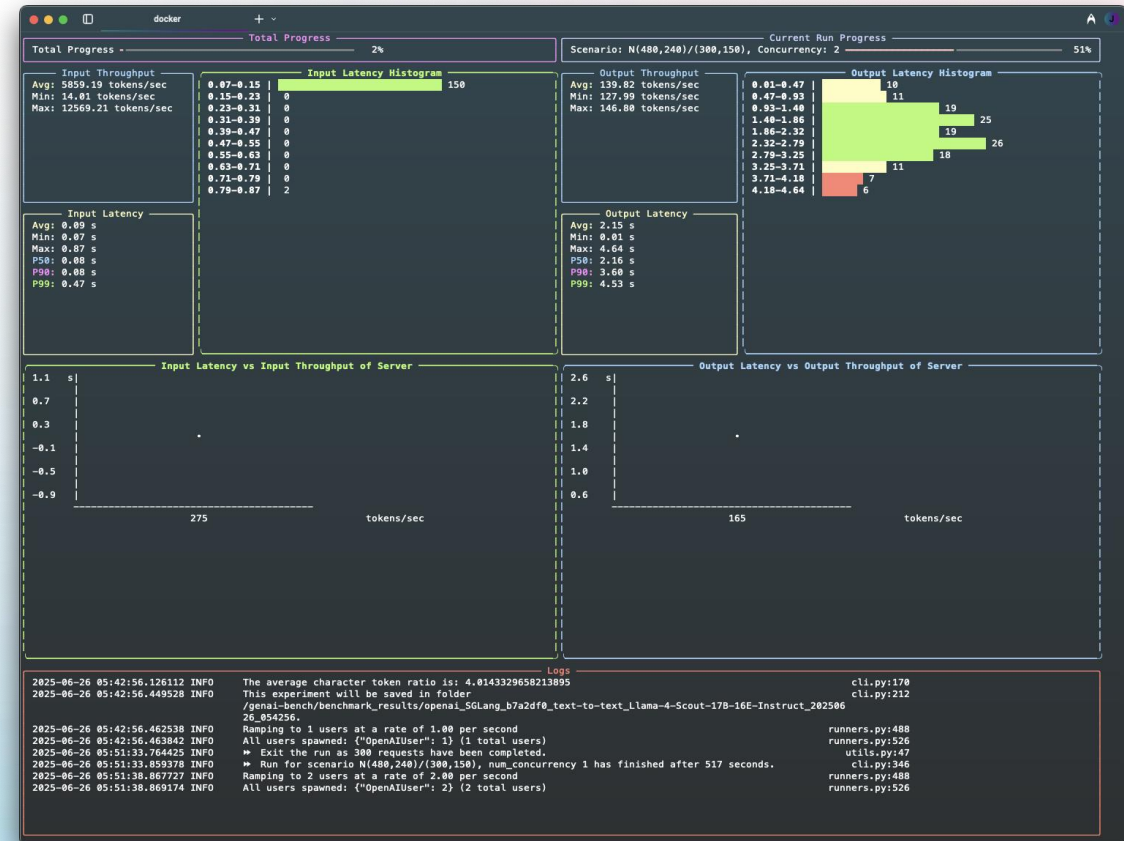
Benchmarking complete.

(Source: <https://github.com/vllm-project/guidellm>)

Genai-bench

Scale Efficiently: Evaluate and Optimize Your LLM Deployments for Real-World Inference

- CLI Tool
- Live UI Dashboard
- Rich Logs
- Experiment Analyzer



(Source: <https://github.com/sgl-project/genai-bench>)

Demo: Benchmarking with Genai-bench

genai-bench.sh

Bash

```
uvx genai-bench benchmark \  
  --api-backend "vllm" \  
  --api-base "http://localhost:8080/" \  
  --api-key "na" \  
  --api-model-name "Qwen/Qwen2.5-7B-Instruct" \  
  --model-tokenizer "Qwen/Qwen2.5-7B-Instruct" \  
  --task "text-to-text" \  
  --max-requests-per-run 1000 \  
  --max-time-per-run 10 \  
  --server-engine "vLLM" \  
  --server-version "v0.7.0" \  
  --server-gpu-count 1
```

Testbed

GPU: 1/4 NVIDIA H20 (w/ HAMi)

Model: Qwen/Qwen2.5-7B-Instruct

Engine: vLLM v0.7.0



<https://github.com/rudeigerc/kcd-hangzhou-and-oicd-2025>

Demo: Benchmarking with Genai-bench

Understanding Parameters

genai-bench.sh

Bash

```
uvx genai-bench benchmark \  
  --api-backend "vllm" \  
  --api-base "http://localhost:8080/" \  
  --api-key "na" \  
  --api-model-name "Qwen/Qwen2.5-7B-Instruct" \  
  --model-tokenizer "Qwen/Qwen2.5-7B-Instruct" \  
  --task "text-to-text" \  
  --max-requests-per-run 1000 \  
  --max-time-per-run 10 \  
  --server-engine "vLLM" \  
  --server-version "v0.7.0" \  
  --server-gpu-count 1
```

text-to-text

Benchmarks generating text output from text input, such as chat or QA tasks.

- N(480,240)/(300,150)
- D(100,100)
- D(100,1000)
- D(2000,200)
- D(7800,200)

Demo: Benchmarking with Genai-bench

Understanding Benchmarking Results

Concurrency

RPS

Throughput

Aggregated Metrics

JSON

```
{
  "scenario": "N(480,240)/(300,150)",
  "num_concurrency": 16,
  "batch_size": 1,
  "iteration_type": "num_concurrency",
  "run_duration": 319.9686911179451,
  "mean_output_throughput_tokens_per_s": 935.4915287310512,
  "mean_input_throughput_tokens_per_s": 1484.4171107507523,
  "mean_total_tokens_throughput_tokens_per_s": 2419.9086394818037,
  "mean_total_chars_per_hour": 35371132.83609123,
  "requests_per_second": 3.125305780718981,
  "error_codes_frequency": {},
  "error_rate": 0,
  "num_error_requests": 0,
  "num_completed_requests": 1000,
  "num_requests": 1000,
  "stats": {}
}
```

Demo: Benchmarking with Genai-bench

Understanding Benchmarking Results

Individual Request Metrics

JSON

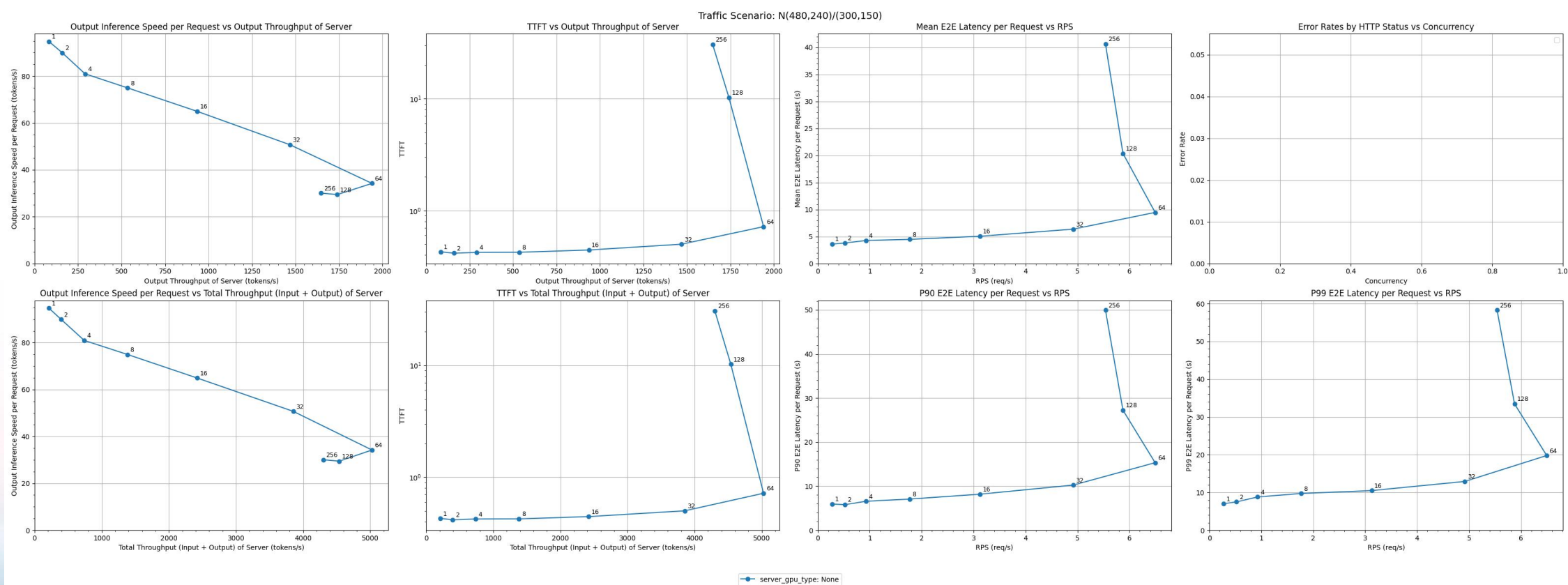
```
{
  "ttft": 2.197405916027492,
  "tpot": 0.07404679199680686,
  "e2e_latency": 2.2714527080242988,
  "output_latency": 0.07404679199680686,
  "output_inference_speed": 13.504973990542673,
  "num_input_tokens": 471,
  "num_output_tokens": 2,
  "total_tokens": 473,
  "input_throughput": 214.3436479189434,
  "output_throughput": 13.504973990542673,
  "error_code": null,
  "error_message": null
}
```

(Source: <https://github.com/sgl-project/genai-bench>)

Glossary	Meaning	Calculation Formula	Units
TTFT	Time to First Token. Initial response time when the first output token is generated. This is also known as the latency for the input (input) stage.	$TTFT = time_at_first_token - start_time$	seconds
End-to-End Latency	End-to-End latency. This metric indicates how long it takes from submitting a query to receiving the full response, including network latencies.	$e2e_latency = end_time - start_time$	seconds
TPOT	Time Per Output Token. The average time between two subsequent generated tokens.	$TPOT = (e2e_latency - TTFT) / (num_output_tokens - 1)$	seconds
Output Latency	Output latency. This metric indicates how long it takes to receive the full response after the first token is generated.	$output_latency = e2e_latency - TTFT$	seconds
Output Inference Speed	The rate of how many tokens the model can generate per second for a single request.	$inference_speed = 1 / TPOT$	tokens/second
Num of Input Tokens	Number of prompt tokens.	$num_input_tokens = tokenizer.encode(prompt)$	tokens
Num of Output Tokens	Number of output tokens.	$num_output_tokens = num_completion_tokens$	tokens
Num of Request Tokens	Total number of tokens processed in one request.	$num_request_tokens = num_input_tokens + num_output_tokens$	tokens
Input Throughput	The overall throughput of input (input process).	$input_throughput = num_input_tokens / TTFT$	tokens/second
Output Throughput	The throughput of output (output generation) for a single request.	$output_throughput = (num_output_tokens - 1) / output_latency$	tokens/second

Demo: Benchmarking with Genai-bench

Understanding Benchmarking Results



Demo: Benchmarking with Genai-bench

User Story: Maximum Concurrency with SLO Attainment

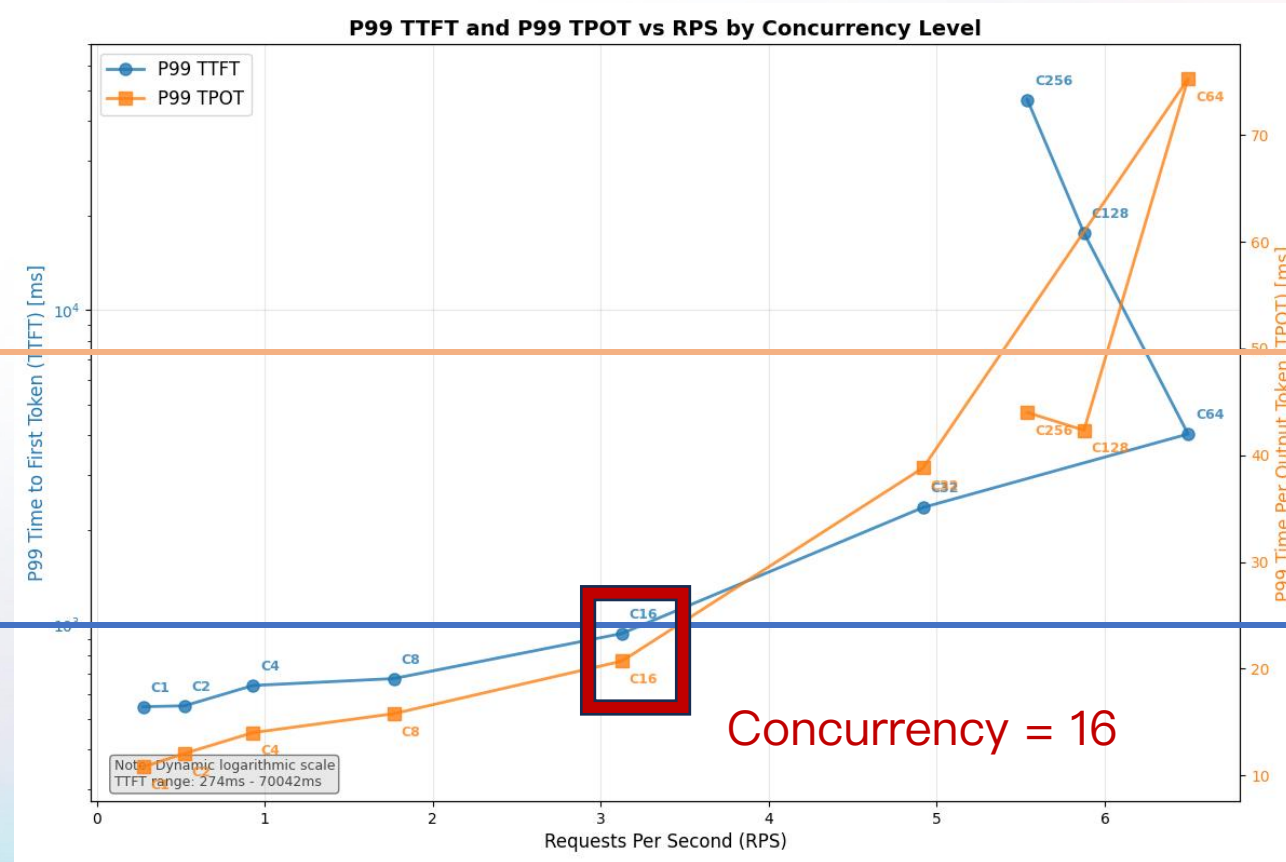
Target

P99 TTFT $\leq 1000\text{ms}$

P99 TPOT $\leq 50\text{ms}$

TTFT

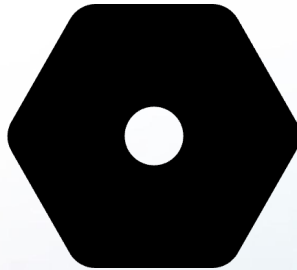
TPOT



Demo: Benchmarking with Genai-bench

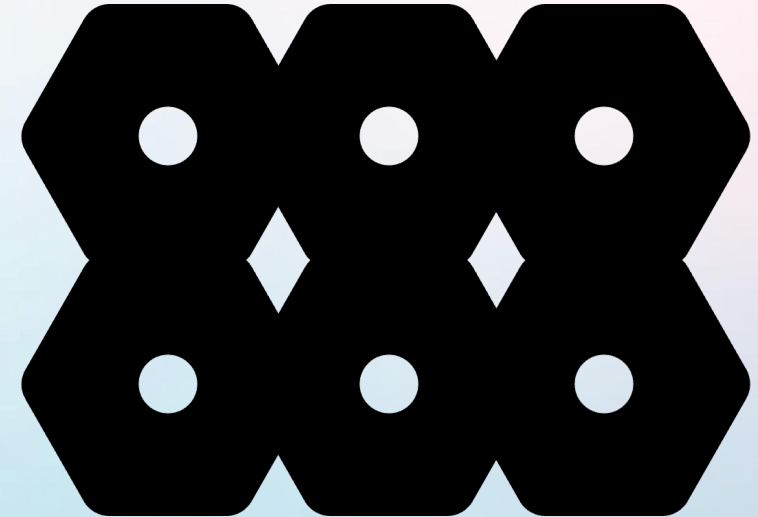
User Story: Maximum Concurrency with SLO Attainment

Concurrency = 16



3 RPS

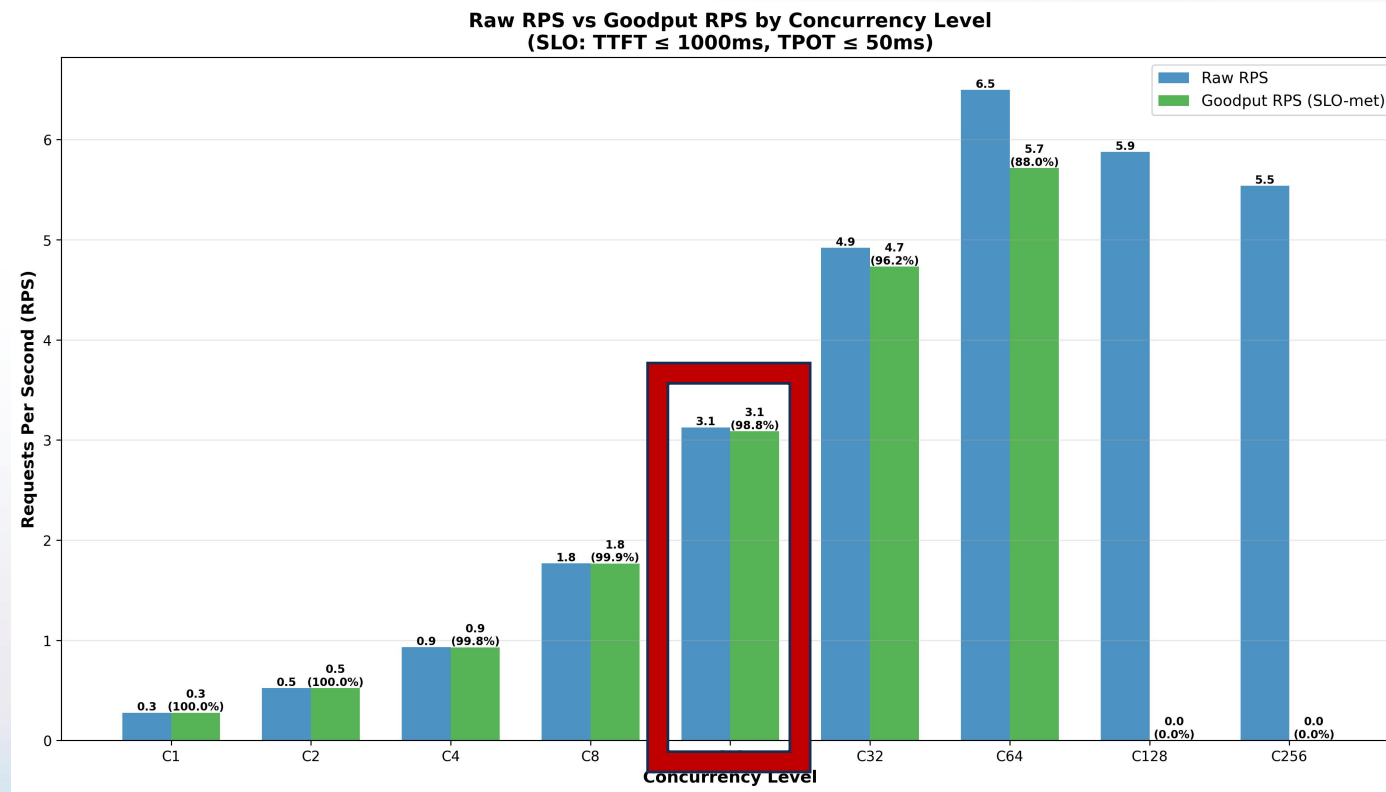
Target = 100 RPS



33 Instances

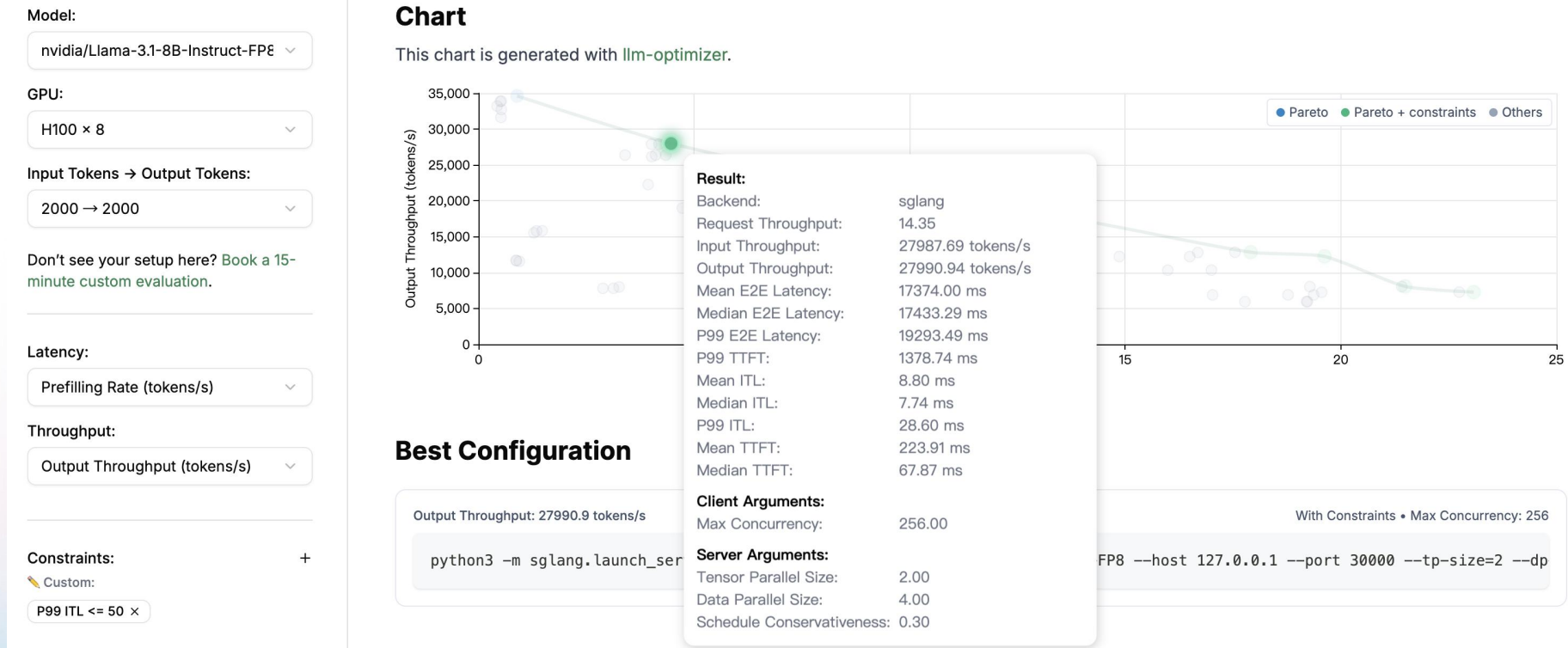
Demo: Benchmarking with Genai-bench

User Story: Throughput v.s. Goodput



Recap: Pareto Frontier

LLM Performance Explorer by BentoML



(Source: <https://www.bentoml.com/llm-perf/> / <https://github.com/bentoml/llm-optimizer>)

Summary

Takeaways and ... More?

Is Goodput All You Need?

Takeaways

Is Goodput All You Need? **Not Exactly.**

- **SLO Heterogeneity**
 - Different workloads claim different SLOs, benchmark with your actual use case
- **Benchmarking Frameworks**
 - Many inference benchmarking frameworks are available
 - Can provide real logs or use public tracing
- **SLO Attainment + Goodput**
 - Balance user experience and cost efficiency

What's More

KubeCon NA 2025



 **Tutorial: A Cross-Industry Benchmarking Tutorial for Distributed LLM Inference on Kubernetes** - Jing Chen, IBM Research; Junchen Jiang, University of Chicago; Ganesh Kudleppanavar, NVIDIA; Samuel Monson, Red Hat; Jason Kramberger, Google

 Tuesday November 11, 2025 4:15pm - 5:30pm EST
 Building B | Level 5 | Thomas Murphy Ballroom 2-3

As organizations deploy LLMs as distributed stacks in production Kubernetes environments, optimizing inference performance has been critical. This collaborative tutorial brings together experts from Google, NVIDIA, RedHat, IBM, and University of Chicago (LMCache) to provide practical benchmarking techniques for impactful LLM optimization strategies.

Using identified use cases as examples, we'll show how to benchmark key optimization strategies: KV Cache offloading, autoscaling, prefix/session-aware routing, KVCache-aware routing, and xPyD for prefill decode disaggregation. Attendees will learn a unified benchmarking approach integrating tools including vLLM, LMBenchmark, GuideLLM, GenAlper, inference-perf, and fmpref. Through live demonstrations, participants gain hands-on experience with production-tested methodologies reflecting real-world scenarios. Attendees will be equipped to implement these approaches for data-driven LLM serving optimizations on Kubernetes.

Speakers



Samuel Monson
Software Engineer, Red Hat



Junchen Jiang
CS Professor, University of Chicago



Ganesh Kudleppanavar
Software Engineering Manager, Nvidia Corporation



Jason Kramberger
Software Engineer, Google



Jing Chen
Software Engineer, IBM Research

<https://sched.co/27FXL>

References

Projects

[sglang/genai-bench](#)

[vllm-project/guidellm](#)

[llm-d/llm-d-benchmark](#)

[ai-dynamo/aiperf](#)

[kubernetes-sigs/inference-perf](#)

[fmperf-project/fmperf](#)

[bentoml/llm-optimizer](#)

[AlBrix Benchmark](#)

[NVIDIA TRI genai-perf](#)

[huggingface/inference-benchmark](#)

[ray-project/llmperf](#)

[fw-ai/benchmark](#)

References

Talks

[A Practical Guide To Benchmarking AI and GPU Workloads in Kubernetes – Yuan Chen, NVIDIA & Chen Wang, IBM Research. KubeCon EU 2025](#)

Publications

Zhong et al. 2024. [DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving](#). OSDI 2024

Agrawal et al. 2024. [Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve](#). OSDI 2024

Hong et al. 2025. [SOLA: Optimizing SLO Attainment for Large Language Model Serving with State-Aware Scheduling](#). MLSys 2025

Wang et al. 2025. [Revisiting Service Level Objectives and System Level Metrics in Large Language Model Serving](#). arXiv:2410.14257

Posts

AI Inference Stack. <https://www.thelis.org/blog/inference-stack>

The AI Inference Stack | Wing Venture Capital. <https://www.wing.vc/content/the-ai-inference-stack>

References

Posts

Throughput is Not All You Need: Maximizing Goodput in LLM Serving using Prefill-Decode Disaggregation. <https://hao-ai-lab.github.io/blogs/distserve/>

Key metrics for LLM inference | LLM Inference Handbook. <https://bentoml.com/llm/inference-optimization/llm-inference-metrics>

LLM performance benchmarks | LLM Inference Handbook. <https://bentoml.com/llm/inference-optimization/llm-performance-benchmarks>

LLM Inference Benchmarking: Fundamental Concepts | NVIDIA Technical Blog. <https://developer.nvidia.com/blog/llm-benchmarking-fundamental-concepts/>

LLM Inference Benchmarking Guide: NVIDIA GenAI-Perf and NIM | NVIDIA Technical Blog. <https://developer.nvidia.com/blog/llm-performance-benchmarking-measuring-nvidia-nim-performance-with-genai-perf/>

LLM Inference Benchmarking: How Much Does Your LLM Inference Cost? | NVIDIA Technical Blog. <https://developer.nvidia.com/blog/llm-inference-benchmarking-how-much-does-your-llm-inference-cost/>

LLM Inference Benchmarking: Performance Tuning with TensorRT-LLM | NVIDIA Technical Blog. <https://developer.nvidia.com/blog/llm-inference-benchmarking-performance-tuning-with-tensorrt-llm/>

Thank you!

郑宇宸 Yu-Chen Cheng

E-mail: rudeigerc@gmail.com

GitHub: [@rudeigerc](https://github.com/rudeigerc)

X: [@yuchenrcheng](https://twitter.com/yuchenrcheng)

Blog: <https://rudeigerc.dev>

